

Chapter 4 SQL: Data Manipulation

Chapter Objectives

In this chapter you will learn:

- The purpose and importance of the Structured Query Language (SQL).
 - The history and development of SQL.
 - How to write an SQL command.
 - How to retrieve data from the database using the SELECT statement.
 - How to build SQL statements that:
 - use the WHERE clause to retrieve rows satisfying various conditions;
 - sort query results using ORDER BY;
 - use the aggregate functions of SQL;
 - group data using GROUP BY;
 - use subqueries;
 - join tables together;
 - perform set operations (UNION, INTERSECT, EXCEPT).
 - How to perform database updates using INSERT, UPDATE, and DELETE.

In Chapters 3 we described the relational data model and relational languages in some detail. A particular language that emerged from the development of the relational model is the Structured Query Language — SQL as it is universally known. Over recent years, SQL has established itself as the standard relational database language. In 1986, the American National Standards Institute (ANSI) defined a standard for SQL, which was subsequently adopted in 1987 as an international standard by the International Organization for Standardization (ISO, 1987). More than one hundred DBMSs now support SQL, running on hardware platforms ranging from personal computers to mainframes.

Given the central importance of SQL, four chapters and an appendix of this book are devoted to an in-depth examination of the language, providing comprehensive coverage for technical and non-technical readers alike — including programmers, database professionals, and managers. Throughout these chapters we concentrate primarily on the ISO definition of SQL. Given the complexity of the standard, however, we do not attempt to cover every aspect of the language. In this chapter, the focus is on the data manipulation statements.

4.1 Introduction to SQL

This section outlines the objectives of SQL, provides a brief history of the language, and explains its importance to database applications.

4.1.1 Objectives of SQL

An ideal database language should enable a user to:

- Create the database and its relation structures.
- Perform fundamental data management tasks — inserting, modifying, and deleting data from relations.
- Perform queries ranging from simple to highly complex.

These capabilities should be achievable with minimal user effort, and the language's syntax and command structure should be relatively straightforward to learn. The language should also be portable — conforming to a recognized standard so that the same command structure and syntax can be used when moving from one DBMS to another. SQL is designed to satisfy all of these requirements.

SQL is a transform-oriented language: a language designed to use relations to transform inputs into required outputs. The ISO SQL standard has two major components:

- A Data Definition Language (DDL) for defining the database structure and controlling access to data.
- A Data Manipulation Language (DML) for retrieving and updating data.

Prior to the 1999 release of the standard (SQL:1999 or SQL3), SQL contained only definitional and manipulative commands — it lacked flow-of-control constructs such as IF...THEN...ELSE, GOTO, or DO...WHILE. Such constructs had to be provided by a surrounding programming or job-control language, or handled interactively by user decisions. Owing to this lack of computational completeness, SQL can be used in two modes: interactively, by entering statements directly at a terminal; or embedded within a procedural language.

SQL is a relatively accessible language for several reasons:

- It is a nonprocedural language — you specify what information you require, not how to retrieve it. SQL does not require the user to specify data access methods.
- Like most modern languages, SQL is essentially free-format: statements do not need to be entered in particular screen positions.
- The command structure is built from standard English words such as CREATE TABLE, INSERT, and SELECT. For example:

```
CREATE TABLE Staff (staffNo VARCHAR(5), IName VARCHAR(15), salary DECIMAL(7,2));
INSERT INTO Staff VALUES ('SG16', 'Brown', 8300);
SELECT staffNo, IName, salary
FROM Staff
WHERE salary > 10000;
```

- SQL can be used productively by a broad range of users including database administrators, managers, application developers, and many types of end-user.

An international standard now exists for SQL, making it both the formal and de facto standard language for defining and manipulating relational databases (ISO, 1992, 2011a).

4.1.2 History of SQL

The history of the relational model — and indirectly of SQL — traces back to the seminal paper published by E. F. Codd during his time at IBM's Research Laboratory in San José (Codd, 1970). In 1974, D. Chamberlin, also at the IBM San José Laboratory, defined a language called the Structured English Query Language, or SEQUEL. A revised version, SEQUEL/2, was defined in 1976, but the name was subsequently changed to SQL for legal reasons (Chamberlin and Boyce, 1974; Chamberlin et al., 1976). Many people still pronounce SQL as "See-Quel," though the official pronunciation is "S-Q-L."

IBM developed a prototype DBMS based on SEQUEL/2 called System R (Astrahan et al., 1976), designed to validate the feasibility of the relational model. Among its many contributions, one of the most significant was the development of SQL itself. It is worth noting, however, that SQL's roots lie in an earlier research language called SQUARE — Specifying Queries As Relational Expressions — which predated the System R project. SQUARE was conceived as a research vehicle for implementing relational algebra using English sentences (Boyce et al., 1975).

In the late 1970s, Oracle Corporation (as it is now known) released the Oracle database system, arguably the first commercial implementation of a relational DBMS based on SQL. INGRES followed shortly afterward with a query language called QUEL, which — though more formally structured than SQL — was less English-like in its syntax. When SQL emerged as the dominant standard for relational systems, INGRES was converted to an SQL-based DBMS. IBM released its first commercial RDBMS, SQL/DS, for the DOS/VSE and VM/CMS environments in 1981 and 1982 respectively, and subsequently DB2 for the MVS environment in 1983.

In 1982, ANSI began work on a Relational Database Language (RDL) based on a concept paper from IBM. ISO joined the effort in 1983, and together the two bodies defined a standard for SQL. (The name RDL was dropped in 1984, and the draft standard was revised to align more closely with existing SQL implementations.)

The initial ISO standard, published in 1987, attracted substantial criticism. The influential researcher C. J. Date argued that important features — including referential integrity constraints and certain relational operators — had been omitted, and that the language was unnecessarily redundant, with multiple ways to express the same query (Date, 1986, 1987a, 1990). Much of the criticism was valid and had in fact been recognized by the standards bodies before publication. The decision was nonetheless made to release a standard as quickly as possible in order to establish a common baseline from which the language and its implementations could evolve, rather than waiting until all desired features could be fully defined and agreed upon.

In 1989, ISO published an addendum defining an "Integrity Enhancement Feature" (ISO, 1989). The first major revision to the standard, known as SQL2 or SQL-92, appeared in 1992

(ISO, 1992). Although some features were defined for the first time in that release, many had already been partially or similarly implemented across various existing SQL products. The next major release, SQL:1999 (ISO, 1999a), introduced additional features to support object-oriented data management, discussed in Chapter 9. Further releases followed in late 2003 (SQL:2003), summer 2008 (SQL:2008), and late 2011 (SQL:2011).

Functionality added by individual vendors beyond what the standard specifies is called an extension. For example, while the standard defines six data types, many implementations supplement this set with additional vendor-specific types. Each implementation of SQL is called a dialect. No two dialects are identical, and currently no dialect fully conforms to the ISO standard. As vendors introduce new capabilities, their dialects continue to diverge. Nevertheless, the core of the SQL language shows signs of increasing standardization: a defined set of features called Core SQL now exists, which a vendor must implement to claim conformance with the standard. Many remaining features are organized into optional packages — for example, packages for object features and for OLAP (OnLine Analytical Processing).

Although SQL originated at IBM, its significance quickly prompted other vendors to develop their own implementations. Today, hundreds of SQL-based products are available, with new ones appearing regularly.

4.1.3 Importance of SQL

SQL is the first — and thus far the only — standard database language to achieve broad, widespread acceptance. The only other standardized database language, the Network Database Language (NDL), based on the CODASYL network model, has attracted minimal support. Virtually every major database vendor now offers products built on SQL or with an SQL interface, and most are represented on at least one of the standards bodies.

The investment in SQL — by both vendors and users — is enormous. It has been incorporated into major application architectures including IBM's Systems Application Architecture (SAA) and has been adopted as the strategic database language by many large and influential organizations, including the Open Group consortium for UNIX standards. SQL has also been designated a Federal Information Processing Standard (FIPS), compliance with which is required for all database management system sales to the U.S. government. The SQL Access Group, a consortium of vendors, defined a set of enhancements to SQL intended to support interoperability across heterogeneous systems.

SQL serves as a definitional tool in other standards and even influences the development of additional standards. Examples include ISO's Information Resource Dictionary System (IRDS) standard and the Remote Data Access (RDA) standard. The language's development is underpinned by substantial academic research, which provides both a theoretical foundation and the techniques required for its successful implementation — most notably in the areas of query optimization, data distribution, and security. Specialized implementations of SQL targeting new markets, such as OnLine Analytical Processing (OLAP), have also emerged.

4.1.4 Terminology

The ISO SQL standard does not use the formal relational terminology of relations, attributes, and tuples. Instead, it employs the terms tables, columns, and rows. Throughout our SQL presentation, we generally follow the ISO terminology. It should also be noted that SQL does not strictly adhere to the relational model as defined in Chapter 3. For example, SQL permits a table produced by a SELECT statement to contain duplicate rows, imposes an ordering on columns, and allows users to specify an ordering for the rows of a result table.

4.2 Writing SQL Commands

This section briefly describes the structure of an SQL statement and introduces the notation used throughout this book to define the format of SQL constructs.

An SQL statement consists of reserved words and user-defined words. Reserved words are fixed elements of the SQL language with predefined meanings. They must be spelled exactly as specified and may not be split across lines. User-defined words are names chosen by the user — according to certain syntactic rules — for database objects such as tables, columns, views, and indexes. Words in a statement are constructed according to a set of syntax rules. Although the standard does not require it, many SQL dialects require a statement terminator at the end of each statement, typically a semicolon (;).

Most components of an SQL statement are case-insensitive, meaning letters may be typed in either uppercase or lowercase. The notable exception is literal character data, which must be entered exactly as it appears in the database. For example, if a surname is stored as "SMITH" and a query searches for "Smith", the row will not be found.

Although SQL is free-format, readability is significantly improved by consistent use of indentation and line breaks. The following conventions are recommended:

- Each clause in a statement should begin on a new line.
- The opening keyword of each clause should be aligned with those of other clauses.
- When a clause consists of multiple parts, each part should appear on a separate line, indented under the clause keyword to make the relationship clear.

Throughout this and the next three chapters, SQL statements are defined using the following extended Backus-Naur Form (BNF) notation:

- Uppercase letters denote reserved words, which must be spelled exactly as shown.
- Lowercase letters denote user-defined words.
- A vertical bar (|) indicates a choice between alternatives; for example, a | b | c.
- Curly braces indicate a required element; for example, {a}.
- Square brackets indicate an optional element; for example, [a].
- An ellipsis (...) indicates optional repetition of an item zero or more times.

For example, the notation:

`{a|b} [, c ...]`

means either `a` or `b`, followed by zero or more repetitions of `c` separated by commas.

In practice, DDL statements are used to establish the database structure — creating tables and defining access controls — while DML statements are used to populate and query those tables. In this chapter, however, we present the DML before the DDL, reflecting the greater relevance of data manipulation to general users.

Here is the reconstructed text:

4.3 Data Manipulation

This section examines the following SQL DML statements:

- **SELECT** — to query data in the database
- **INSERT** — to insert data into a table
- **UPDATE** — to update data in a table
- **DELETE** — to delete data from a table

Given the complexity of the **SELECT** statement relative to the other three, the bulk of this section is devoted to **SELECT** and its many variations. We begin with simple queries and progressively introduce additional complexity, showing how sorting, grouping, aggregation, and multi-table queries are constructed. The section concludes with the **INSERT**, **UPDATE**, and **DELETE** statements.

All SQL examples use the DreamHome case study instance shown below, which consists of the following tables:

Branch	(<u>branchNo</u> , street, city, postcode)
Staff	(<u>staffNo</u> , fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent	(<u>propertyNo</u> , street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)
Client	(<u>clientNo</u> , fName, lName, telNo, prefType, maxRent, eMail)
PrivateOwner	(<u>ownerNo</u> , fName, lName, address, telNo, eMail, password)
Viewing	(<u>clientNo</u> , propertyNo, viewDate, comment)

Literals

Before examining the DML statements, it is important to understand literals — the constants used within SQL statements. Every data type supported by SQL has its own literal form. For

practical purposes, the key distinction is that all non-numeric data values must be enclosed in single quotes, while all numeric values must not be. For example, when inserting data:

```
INSERT INTO PropertyForRent(propertyNo, street, city, postcode, type, rooms, rent,
                             ownerNo, staffNo, branchNo)
VALUES ('PA14', '16 Holhead', 'Aberdeen', 'AB7 5SU', 'House', 6, 650.00,
         'CO46', 'SA9', 'B007');
```

The value in the `rooms` column is an integer literal and the value in `rent` is a decimal literal — neither is enclosed in quotes. All other columns hold character strings and are quoted accordingly.

4.3.1 Simple Queries

The purpose of the SELECT statement is to retrieve and display data from one or more database tables. It is an exceptionally powerful command, capable of performing the equivalent of the relational algebra's Selection, Projection, and Join operations within a single statement (see Section 5.1). SELECT is the most frequently used SQL command. Its general form is:

SELECT	[DISTINCT ALL] { * [columnExpression [AS newName]] [, . . .] }
FROM	TableName [alias] [, . . .]
[WHERE	condition]
[GROUP BY	columnList] [HAVING condition]
[ORDER BY	columnList]

Here, `columnExpression` represents a column name or an expression; `TableName` is the name of an existing database table or view the user has access to; and `alias` is an optional abbreviation for `TableName`. The logical sequence in which a SELECT statement is processed is:

FROM	specifies the table or tables to be used
WHERE	filters the rows subject to some condition
GROUP BY	forms groups of rows with the same column value
HAVING	filters the groups subject to some condition
SELECT	specifies which columns are to appear in the output
ORDER BY	specifies the order of the output

The order of clauses in the SELECT statement is fixed and cannot be changed. Only SELECT and FROM are mandatory; all remaining clauses are optional. The SELECT operation is closed — the result of any query on a table is itself a table (see Section 5.1).

Retrieve all rows

EXAMPLE 4.1 — Retrieve all columns, all rows

List full details of all staff.

Since no restrictions are specified and all columns are required, the WHERE clause is unnecessary:

```
SELECT staffNo, fName, lName, position, sex, DOB, salary, branchNo  
  
FROM Staff;
```

Because retrieving all columns is a common requirement, SQL provides a shorthand: an asterisk (*) in place of the column list. The following query is equivalent:

```
SELECT *  
  
FROM Staff;
```

The result is shown in Table 4.1.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

EXAMPLE 4.2 — Retrieve specific columns, all rows

Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary  
  
FROM Staff;
```

This creates a result table derived from Staff containing only the four specified columns in the stated order (Table 4.2). Unless an ORDER BY clause is specified, the row ordering is not guaranteed — though some DBMSs may impose a default order based on the primary key.

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG37	Ann	Beech	12000.00
SG14	David	Ford	18000.00
SA9	Mary	Howe	9000.00
SG5	Susan	Brand	24000.00
SL41	Julie	Lee	9000.00

EXAMPLE 4.3 — Use of DISTINCT

List the property numbers of all properties that have been viewed.

```
SELECT propertyNo
```

```
FROM Viewing;
```

propertyNo
PA14
PG4
PG4
PA14
PG36

The result (Table 4.3(a)) contains duplicates, because unlike the relational algebra Projection operation, SQL's SELECT does not automatically eliminate duplicate rows. To remove them, the DISTINCT keyword is used:

```
SELECT DISTINCT propertyNo
```

```
FROM Viewing;
```

This produces the deduplicated result shown in Table 4.3(b).

propertyNo

PA14

PG4

PG36

EXAMPLE 4.4 — Calculated fields

Produce a list of monthly salaries for all staff, showing the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary/12
```

```
FROM Staff;
```

This is almost identical to Example 4.2, except that monthly salary is computed by dividing the annual salary by 12, producing the result in Table 4.4. This is an example of a calculated (or derived) field. SQL expressions in the SELECT list may use addition, subtraction, multiplication, and division, with parentheses for complex expressions. Any columns referenced in arithmetic expressions must be of a numeric type.

The fourth column has no name because SQL cannot infer one from an expression. Some dialects assign a positional name (e.g., `col4`); others leave the heading blank. The ISO standard allows the AS clause to assign a name:

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary
```

```
FROM Staff;
```

The column heading in the result table would then be `monthlySalary`.

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

Row selection (WHERE clause)

The previous examples retrieve all rows. To restrict retrieval to specific rows, the WHERE clause is used. The WHERE clause consists of the keyword WHERE followed by a search condition. The five basic types of search condition (predicates in ISO terminology) are:

- Comparison — compares the value of one expression with another.
- Range — tests whether a value falls within a specified range.
- Set membership — tests whether a value equals one of a set of values.
- Pattern match — tests whether a string matches a specified pattern.
- Null — tests whether a column value is null.

The WHERE clause is the SQL equivalent of the relational algebra Selection operation.

EXAMPLE 4.5 — Comparison search condition

List all staff with a salary greater than £10,000.

```
SELECT staffNo, fName, lName, position, salary  
  
FROM Staff  
  
WHERE salary > 10000;
```

The result (Table 4.5) contains only those Staff rows satisfying the condition **salary > 10000**.

SQL's basic comparison operators are: **=**, **<>** (ISO) or **!=** (some dialects), **<**, **<=**, **>**, and **>=**. More complex predicates can be constructed using the logical operators AND, OR, and NOT, with parentheses to control evaluation order. The evaluation rules are:

- Expressions evaluate left to right.
- Parenthesized sub-expressions evaluate first.
- NOT is evaluated before AND and OR.
- AND is evaluated before OR.

Use of parentheses is always recommended to eliminate ambiguity.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

EXAMPLE 4.6 — Compound comparison search condition

List the addresses of all branch offices in London or Glasgow.

```
SELECT *
```

```
FROM Branch
```

```
WHERE city = 'London' OR city = 'Glasgow';
```

The result (Table 4.6) includes branches in both cities.

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

EXAMPLE 4.7 — Range search condition (BETWEEN/NOT BETWEEN)

List all staff with a salary between £20,000 and £30,000.

```
SELECT staffNo, fName, lName, position, salary
```

```
FROM Staff
```

```
WHERE salary BETWEEN 20000 AND 30000;
```

BETWEEN is inclusive — staff earning exactly £20,000 or £30,000 are included (Table 4.7). NOT BETWEEN tests for values outside the range. This query can equally be expressed as:

```
WHERE salary >= 20000 AND salary <= 30000
```

BETWEEN is simply a more concise way to express a range condition.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

EXAMPLE 4.8 — Set membership search condition (IN/NOT IN)

List all managers and supervisors.

```
SELECT staffNo, fName, lName, position
```

```
FROM Staff
```

```
WHERE position IN ('Manager', 'Supervisor');
```

The IN test checks whether a value matches any member of the listed set (Table 4.8). NOT IN checks for values absent from the set. This query is equivalent to:

```
WHERE position = 'Manager' OR position = 'Supervisor'
```

IN is more concise, especially when the set contains many values.

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

EXAMPLE 4.9 — Pattern match search condition (LIKE/NOT LIKE)

Find all owners with the string 'Glasgow' in their address.

SQL supports two pattern-matching wildcards:

- % — represents any sequence of zero or more characters.
- _ — represents any single character.

All other characters match themselves. Examples:

- `address LIKE 'H%'` — first character must be H, rest can be anything.
- `address LIKE 'H___'` — exactly four characters, first must be H.
- `address LIKE '%e'` — any sequence ending in e.
- `address LIKE '%Glasgow%'` — any string containing 'Glasgow'.
- `address NOT LIKE 'H%'` — first character cannot be H.

To match a literal wildcard character, an escape character is used:

`LIKE '15#%' ESCAPE '#'`

The query is:

`SELECT ownerNo, fName, IName, address, telNo`

`FROM PrivateOwner`

`WHERE address LIKE '%Glasgow%';`

The result is shown in Table 4.9. Note that some RDBMSs (such as Microsoft Office Access) use `*` and `?` instead of `%` and `_`.

ownerNo	fName	IName	address	telNo
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

EXAMPLE 4.10 — NULL search condition (IS NULL/IS NOT NULL)

List the details of all viewings on property PG4 where a comment has not been supplied.

A null cannot be compared using `=` or `<>` — such comparisons always return false for null values. Nulls must be tested explicitly with `IS NULL`:

`SELECT clientNo, viewDate`

`FROM Viewing`

`WHERE propertyNo = 'PG4' AND comment IS NULL;`

The result (Table 4.10) shows the viewing with no comment. `IS NOT NULL` tests for non-null values.

clientNo

viewDate

CR56

26-May-13

4.3.2 Sorting Results (ORDER BY Clause)

By default, rows in an SQL result table are not guaranteed to appear in any particular order. The ORDER BY clause allows the result to be sorted. It consists of a list of column identifiers — either column names or column position numbers — separated by commas. Each column can be sorted in ascending (ASC, the default) or descending (DESC) order. ORDER BY must always be the last clause in a SELECT statement.

EXAMPLE 4.11 — Single-column ordering

Produce a list of salaries for all staff, arranged in descending order of salary.

```
SELECT staffNo, fName, lName, salary
```

```
FROM Staff
```

```
ORDER BY salary DESC;
```

The result (Table 4.11) is sorted from highest to lowest salary. This could equivalently be written as **ORDER BY 4 DESC**, referring to the fourth column in the SELECT list.

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG5	Susan	Brand	24000.00
SG14	David	Ford	18000.00
SG37	Ann	Beech	12000.00
SA9	Mary	Howe	9000.00
SL41	Julie	Lee	9000.00

EXAMPLE 4.12 — Multiple column ordering

Produce an abbreviated list of properties arranged in order of property type.

```
SELECT propertyNo, type, rooms, rent  
FROM PropertyForRent  
ORDER BY type;
```

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

Without a secondary sort key, flats of the same type appear in arbitrary order (Table 4.12(a)). To order by rent within each type:

```
SELECT propertyNo, type, rooms, rent  
FROM PropertyForRent  
ORDER BY type, rent DESC;
```

The result (Table 4.12(b)) is sorted first by property type ascending (the default), then within each type by rent descending.

The ISO standard specifies that nulls in an ORDER BY column should be treated as either greater or less than all non-null values; the choice is left to the DBMS implementor.

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

4.3.3 Using the SQL Aggregate Functions

Beyond retrieving individual rows and columns, it is often necessary to summarize or aggregate data — analogous to the totals that appear at the bottom of a printed report. The ISO standard defines five aggregate functions:

- COUNT — returns the number of values in a specified column.
- SUM — returns the sum of the values in a specified column.
- AVG — returns the average of the values in a specified column.
- MIN — returns the smallest value in a specified column.
- MAX — returns the largest value in a specified column.

Each function operates on a single column and returns a single value. COUNT, MIN, and MAX apply to numeric and non-numeric data; SUM and AVG apply to numeric data only. With the exception of COUNT(), *all aggregate functions eliminate null values before computing*. COUNT() is a special form that counts all rows regardless of nulls or duplicate values.

To eliminate duplicates before applying a function, use the DISTINCT keyword before the column name. The ISO standard also allows ALL (the default) when duplicates should be retained. DISTINCT has no effect on MIN and MAX, but may affect SUM and AVG. DISTINCT can be specified only once per query.

Aggregate functions may only appear in the SELECT list and in the HAVING clause — not in a WHERE clause or anywhere else. When an aggregate function appears in the SELECT list without a GROUP BY clause, no other column references are permitted in the SELECT list unless they are arguments to an aggregate function. For example, the following is illegal:

```
SELECT staffNo, COUNT(salary)
```

```
FROM Staff;
```

because `staffNo` appears outside an aggregate function without a GROUP BY clause.

EXAMPLE 4.13 — Use of COUNT(*)

How many properties cost more than £350 per month to rent?

```
SELECT COUNT(*) AS myCount  
FROM PropertyForRent  
WHERE rent > 350;
```

The WHERE clause restricts the query to qualifying properties; COUNT then totals them. The result is shown in Table 6.13.

EXAMPLE 4.14 — Use of COUNT(DISTINCT)

How many different properties were viewed in May 2013?

```
SELECT COUNT(DISTINCT propertyNo) AS myCount  
FROM Viewing  
WHERE viewDate BETWEEN '1-May-13' AND '31-May-13';
```

DISTINCT eliminates duplicate property numbers before counting, since the same property may be viewed multiple times. The result is shown in Table 6.14.

EXAMPLE 4.15 — Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
WHERE position = 'Manager';
```

The WHERE clause restricts the computation to Manager rows, and the two aggregate functions are applied to this subset. The result is shown in Table 4.15.

myCount	mySum
2	54000.00

EXAMPLE 4.16 — Use of MIN, MAX, AVG

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS myMin, MAX(salary) AS myMax, AVG(salary) AS myAvg  
FROM Staff;
```

No WHERE clause is needed since all staff are included. The result is shown in Table 4.16.

myMin	myMax	myAvg
9000.00	30000.00	17000.00

4.3.4 Grouping Results (GROUP BY Clause)

The aggregate queries in the previous section condense all data into a single summary row. Often, however, subtotals for subgroups are more useful. The GROUP BY clause groups rows that share the same values in specified columns, producing one summary row per group. These columns are called grouping columns. The ISO standard requires that when GROUP BY is used, every item in the SELECT list must be single-valued per group. Specifically, the SELECT clause may contain only:

- column names (which must also appear in the GROUP BY clause unless used only within aggregate functions);
- aggregate functions;
- constants;
- expressions combining these.

When WHERE and GROUP BY are both used, the WHERE clause is applied first to filter individual rows; then the remaining rows are grouped. For GROUP BY purposes, the ISO standard treats two null values as equal — rows with the same null grouping column values are placed in the same group.

EXAMPLE 4.17 — Use of GROUP BY

Find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
GROUP BY branchNo
```

ORDER BY branchNo;

staffNo and **salary** appear only inside aggregate functions so they do not need to be in the GROUP BY list. **branchNo** appears unaggregated in the SELECT list and must therefore appear in GROUP BY. The result (Table 4.17) shows one row per branch.

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

Conceptually, SQL processes this query by: (1) dividing staff rows into groups by **branchNo**; (2) computing COUNT and SUM for each group; and (3) sorting the result by **branchNo**.

An alternative formulation using scalar subqueries in the SELECT list is also possible:

SELECT branchNo,

(SELECT COUNT(staffNo) FROM Staff s WHERE s.branchNo = b.branchNo),

(SELECT SUM(salary) FROM Staff s WHERE s.branchNo = b.branchNo)

FROM Branch b

ORDER BY branchNo;

This version, however, produces results for every branch in the Branch table — potentially including branches with no staff.

branchNo	staffNo	salary		COUNT(staffNo)	SUM(salary)
B003	SG37	12000.00	}	3	54000.00
B003	SG14	18000.00			
B003	SG5	24000.00			
B005	SL21	30000.00	}	2	39000.00
B005	SL41	9000.00			
B007	SA9	9000.00	}	1	9000.00

Restricting groupings (HAVING clause)

The HAVING clause filters groups in the same way that WHERE filters individual rows. It is used with GROUP BY to exclude groups from the final result. While WHERE cannot include aggregate functions, HAVING can — and in practice, the HAVING search condition almost always includes at least one aggregate function. Any condition expressible without an aggregate function should be placed in the WHERE clause instead.

The HAVING clause is not strictly necessary — any query using it can be rewritten without it — but it provides a clear and natural way to express group-level restrictions.

EXAMPLE 4.18 — Use of HAVING

For each branch office with more than one member of staff, find the number of staff and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
HAVING COUNT(staffNo) > 1
ORDER BY branchNo;
```

The HAVING clause removes any group with only one staff member. The result is shown in Table 4.18.

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

4.3.5 Subqueries

A complete SELECT statement can be embedded within another SELECT statement. The result of this inner SELECT (or subselect) is used by the outer statement. When a subselect appears in the WHERE or HAVING clause of an outer SELECT, it is called a subquery or nested query. Subselects can also appear in INSERT, UPDATE, and DELETE statements (Section 4.3.10). There are three types of subquery:

- A scalar subquery returns a single column and a single row — a single value. It can be used wherever a single value is expected.
- A row subquery returns multiple columns but a single row. It is used where a row value

constructor is needed.

- A table subquery returns one or more columns and multiple rows. It is used wherever a table is expected, such as with the IN predicate.

EXAMPLE 4.19 — Using a subquery with equality

List the staff who work in the branch at '163 Main St'.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (SELECT branchNo
                  FROM Branch
                  WHERE street = '163 Main St');
```

The inner SELECT retrieves the branch number corresponding to the street address — a scalar subquery returning a single value, 'B003'. The outer SELECT then retrieves all staff for that branch. The result is shown in Table 4.19.

A subquery can follow any relational operator (=, <, >, <=, >=, <>) in a WHERE or HAVING clause. The subquery must always appear on the right-hand side of the comparison and must be enclosed in parentheses.

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

EXAMPLE 4.20 — Using a subquery with an aggregate function

List all staff whose salary is greater than the average salary, and show by how much.

```
SELECT staffNo, fName, lName, position,
       salary - (SELECT AVG(salary) FROM Staff) AS salDiff
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

Aggregate functions cannot be used directly in the WHERE clause, so a subquery is used to compute the average. The subquery returns £17,000; the outer query then filters and projects accordingly. The result is shown in Table 4.20.

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

The following rules apply to subqueries:

1. ORDER BY may not be used inside a subquery (only in the outermost SELECT).
2. A subquery's SELECT list must consist of a single column name or expression, except when EXISTS is used.
3. Column names in a subquery refer to the FROM table of the subquery by default; outer table columns must be qualified.
4. A subquery must appear on the right-hand side of any comparison operator.

EXAMPLE 4.21 — Nested subqueries: use of IN

List the properties handled by staff who work in the branch at '163 Main St'.

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
```

```
FROM PropertyForRent
```

```
WHERE staffNo IN (SELECT staffNo
```

```
    FROM Staff
```

```
    WHERE branchNo = (SELECT branchNo
```

```
        FROM Branch
```

```
        WHERE street = '163 Main St'));
```

Working outward from the innermost query: the innermost SELECT returns branch number 'B003'; the middle SELECT returns all staff numbers at that branch (potentially multiple); the outermost SELECT retrieves the properties managed by those staff using IN. The result is shown in Table 4.21.

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

4.3.6 ANY and ALL

The ANY and ALL keywords modify comparisons involving subqueries that return a single column of values.

- ALL: the condition is true only if it is satisfied for every value in the subquery result.
- ANY (or SOME): the condition is true if it is satisfied for at least one value in the subquery result.

If the subquery returns an empty set, ALL yields true and ANY yields false.

EXAMPLE 4.22 — Use of ANY/SOME

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > SOME (SELECT salary
                      FROM Staff
                      WHERE branchNo = 'B003');
```

The inner query returns {12000, 18000, 24000}. The outer query selects staff whose salary exceeds at least one of these values — that is, whose salary exceeds the minimum (12000). The result is shown in Table 4.22.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

EXAMPLE 4.23 — Use of ALL

Find all staff whose salary is larger than the salary of every member of staff at branch B003.

```
SELECT staffNo, fName, IName, position, salary
FROM Staff
WHERE salary > ALL (SELECT salary
                    FROM Staff
                    WHERE branchNo = 'B003');
```

ALL requires the condition to hold for every value in the subquery result — effectively, salary must exceed the maximum of {12000, 18000, 24000}, which is 24000. The result is shown in Table 4.23.

staffNo	fName	IName	position	salary
SL21	John	White	Manager	30000.00

4.3.7 Multi-table Queries

The queries examined so far are restricted to columns from a single table. Many practical queries require data from multiple tables. To combine columns from several tables, a join operation is used. A join forms pairs of related rows from two tables where matching columns in each table share the same value. To execute a join in SQL, multiple table names are listed in the FROM clause separated by commas, with a WHERE clause specifying the matching columns. Table aliases can be assigned to disambiguate column references and as a shorthand notation.

If the final result table must include columns from different tables, a join must be used. If only columns from one table are needed but a filter condition requires data from another, a subquery may suffice.

EXAMPLE 4.24 — Simple join

List the names of all clients who have viewed a property, along with any comments supplied.

```
SELECT c.clientNo, fName, IName, propertyNo, comment
```

```
FROM Client c, Viewing v
```

```
WHERE c.clientNo = v.clientNo;
```

Both Client and Viewing columns are required, so a join is used. The `clientNo` column exists in both tables, so it must be qualified. Here, `c` is the alias for Client and `v` for Viewing. The WHERE clause matches rows on equal `clientNo` values — equivalent to the relational algebra Equijoin. The result is shown in Table 4.24.

clientNo	fName	lName	propertyNo	comment
CR56	Aline	Stewart	PG36	
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR62	Mary	Tregear	PA14	no dining room
CR76	John	Kay	PG4	too remote

The ISO standard also allows alternative join syntax:

```
FROM Client c JOIN Viewing v ON c.clientNo = v.clientNo
```

```
FROM Client JOIN Viewing USING clientNo
```

```
FROM Client NATURAL JOIN Viewing
```

The first alternative produces a result with two `clientNo` columns; the remaining two produce a single merged column.

EXAMPLE 4.25 — Sorting a join

For each branch office, list the staff who manage properties and the properties they manage.

```
SELECT s.branchNo, s.staffNo, fName, lName, propertyNo
```

```
FROM Staff s, PropertyForRent p
```

```
WHERE s.staffNo = p.staffNo
```

```
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

Staff and PropertyForRent are joined on `staffNo`. The result (Table 4.25) is sorted by branch number, then staff number, then property number.

branchNo	staffNo	fName	lName	propertyNo
B003	SG14	David	Ford	PG16
B003	SG37	Ann	Beech	PG21
B003	SG37	Ann	Beech	PG36
B005	SL41	Julie	Lee	PL94
B007	SA9	Mary	Howe	PA14

EXAMPLE 4.26 — Three-table join

For each branch, list the staff who manage properties, the city of the branch, and the properties managed.

```
SELECT b.branchNo, b.city, s.staffNo, fName, lName, propertyNo
FROM Branch b, Staff s, PropertyForRent p
WHERE b.branchNo = s.branchNo AND s.staffNo = p.staffNo
ORDER BY b.branchNo, s.staffNo, propertyNo;
```

branchNo	city	staffNo	fName	lName	propertyNo
B003	Glasgow	SG14	David	Ford	PG16
B003	Glasgow	SG37	Ann	Beech	PG21
B003	Glasgow	SG37	Ann	Beech	PG36
B005	London	SL41	Julie	Lee	PL94
B007	Aberdeen	SA9	Mary	Howe	PA14

Three tables are joined: Branch to Staff on **branchNo**, and Staff to PropertyForRent on **staffNo**. The result is shown in Table 4.26. The ISO standard alternative join syntax for this query is:

```
FROM (Branch b JOIN Staff s USING branchNo) AS bs
JOIN PropertyForRent p USING staffNo
```

EXAMPLE 4.27 — Multiple grouping columns

Find the number of properties handled by each staff member, along with their branch number.

```
SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo
ORDER BY s.branchNo, s.staffNo;
```

Staff and PropertyForRent are joined on **staffNo** to identify only those staff who manage properties. The result is then grouped by both **branchNo** and **staffNo**, counting properties per group. The result is shown in Table 4.27(a).

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

Computing a join

A join is a subset of the more general Cartesian product of two tables, which contains all possible row pairings from both tables. The column set of the product is all columns of the first table followed by all columns of the second. A SELECT statement listing two tables in FROM without a WHERE clause produces the Cartesian product. The ISO standard provides explicit syntax for this:

```
SELECT [DISTINCT | ALL] {* | columnList}
FROM TableName1 CROSS JOIN TableName2
```

Conceptually, a join is computed as follows:

1. Form the Cartesian product of the tables in the FROM clause.
2. Apply the WHERE clause to retain only rows satisfying the join condition (relational algebra restriction).
3. For each remaining row, evaluate each item in the SELECT list (relational algebra projection).
4. If SELECT DISTINCT is specified, eliminate duplicate rows.

5. If ORDER BY is specified, sort the result.

Outer joins

In a standard (inner) join, rows from one table that have no matching row in the other are excluded from the result. The ISO standard provides outer join operators that retain unmatched rows. To illustrate, consider simplified tables Branch1 and PropertyForRent1:

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

The inner join on city:

```
SELECT b.*, p.*
```

```
FROM Branch1 b, PropertyForRent1 p
```

```
WHERE b.bCity = p.pCity;
```

produces two matched rows (Glasgow and London), omitting Bristol and Aberdeen entirely.

EXAMPLE 4.28 — Left Outer join

List all branch offices and any properties in the same city.

```
SELECT b.*, p.*
```

```
FROM Branch1 b LEFT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

The Left Outer join retains all rows from the left (Branch1) table, filling right-table columns with NULL where no match exists. The result (Table 4.28) includes the unmatched Bristol branch with NULLs.

Branch1		PropertyForRent1	
branchNo	bCity	propertyNo	pCity
B003	Glasgow	PA14	Aberdeen
B004	Bristol	PL94	London
B002	London	PG4	Glasgow

EXAMPLE 4.29 — Right Outer join

List all properties and any branch offices in the same city.

```
SELECT b.*, p.*
```

```
FROM Branch1 b RIGHT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

The Right Outer join retains all rows from the right (PropertyForRent1) table, with NULLs where no match exists in Branch1. The result (Table 4.29) includes the Aberdeen property with NULLs.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

EXAMPLE 4.30 — Full Outer join

List branch offices and properties in the same city, including all unmatched rows from either table.

```
SELECT b.*, p.*
```

```
FROM Branch1 b FULL JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

The Full Outer join retains unmatched rows from both tables, filling missing columns with NULLs. The result (Table 4.30) includes both the Bristol branch and the Aberdeen property.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

4.3.8 EXISTS and NOT EXISTS

EXISTS and NOT EXISTS are used exclusively with subqueries and return a simple boolean result. EXISTS is true if and only if the subquery returns at least one row; NOT EXISTS is the inverse. Because only the existence (not the values) of rows matters, the subquery can select any columns, and it is common practice to write:

sql

(SELECT * FROM ...)

EXAMPLE 4.31 — Query using EXISTS

Find all staff who work in a London branch office.

```
SELECT staffNo, fName, lName, position
```

```
FROM Staff s
```

```
WHERE EXISTS (SELECT *
```

```
    FROM Branch b
```

```
    WHERE s.branchNo = b.branchNo AND city = 'London');
```

The subquery checks whether a Branch row exists with the staff member's branch number and city equal to 'London'. The correlated condition `s.branchNo = b.branchNo` is essential — without it, the subquery would be non-correlated and would return true for all staff if any London branch exists, listing every staff member regardless of their branch. The result is shown in Table 4.31. An equivalent join formulation is:

```
SELECT staffNo, fName, lName, position
```

```
FROM Staff s, Branch b
```

WHERE s.branchNo = b.branchNo AND city = 'London';

staffNo	fName	lName	position
SL21	John	White	Manager
SL41	Julie	Lee	Assistant

4.3.9 Combining Result Tables (UNION, INTERSECT, EXCEPT)

SQL supports the standard set operations to merge the results of two or more queries into a single result table:

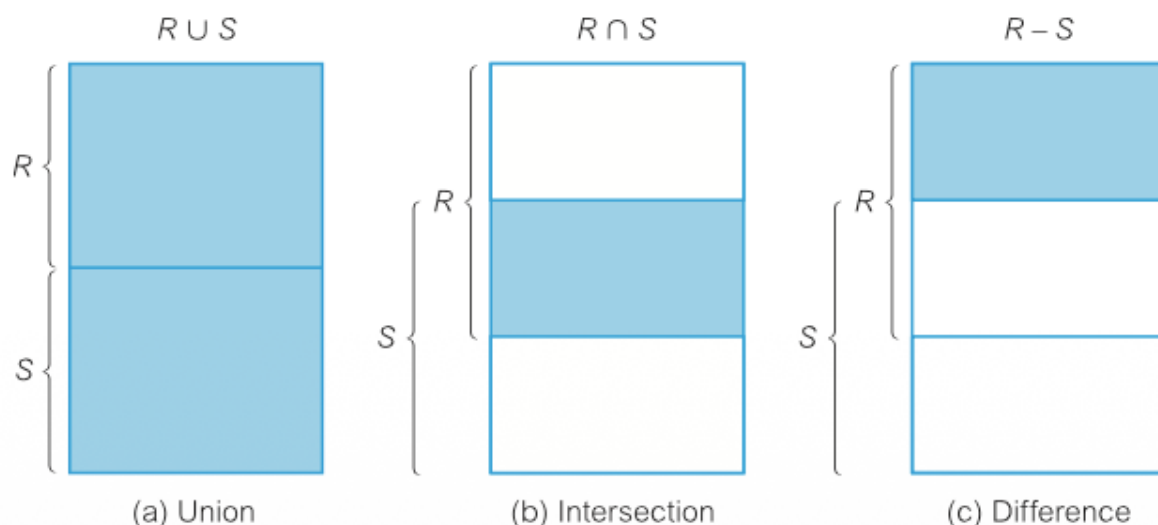
- Union of tables A and B: all rows appearing in either A, B, or both.
- Intersection of tables A and B: only rows appearing in both A and B.
- Difference of tables A and B: rows that appear in A but not in B.

Both operand tables must be union-compatible: the same number of columns, with corresponding columns having the same data types and lengths. The ISO operators are UNION, INTERSECT, and EXCEPT. Some dialects do not support INTERSECT and EXCEPT; others use MINUS in place of EXCEPT.

The general syntax for each set operator is:

operator [ALL] [CORRESPONDING [BY {column1 [, ...]}]]

If CORRESPONDING BY is specified, the operation is performed on the named columns. If ALL is included, duplicate rows are retained in the result.



EXAMPLE 4.32 — Use of UNION

Construct a list of all cities where there is either a branch office or a property.

```
(SELECT city FROM Branch WHERE city IS NOT NULL)
```

```
UNION
```

```
(SELECT city FROM PropertyForRent WHERE city IS NOT NULL);
```

The two result sets are merged and duplicates removed. The result (Table 4.32) lists all distinct cities from either table.

EXAMPLE 4.33 — Use of INTERSECT

Construct a list of all cities where there is both a branch office and a property.

```
(SELECT city FROM Branch)
```

```
INTERSECT
```

```
(SELECT city FROM PropertyForRent);
```

Only cities present in both result sets are retained (Table 4.33). Equivalent formulations without INTERSECT include:

```
SELECT DISTINCT b.city
```

```
FROM Branch b, PropertyForRent p
```

```
WHERE b.city = p.city;
```

and:

```
SELECT DISTINCT city
```

```
FROM Branch b
```

```
WHERE EXISTS (SELECT * FROM PropertyForRent p WHERE b.city = p.city);
```

The availability of multiple equivalent formulations is one of the noted redundancies in SQL.

EXAMPLE 4.34 — Use of EXCEPT

Construct a list of all cities where there is a branch office but no properties.

```
(SELECT city FROM Branch)
```

```
EXCEPT
```

```
(SELECT city FROM PropertyForRent);
```

Only cities in the first result but not the second are retained. Equivalent alternatives include:

```
SELECT DISTINCT city FROM Branch
```

```
WHERE city NOT IN (SELECT city FROM PropertyForRent);
```

and:

```
SELECT DISTINCT city FROM Branch b
```

```
WHERE NOT EXISTS (SELECT * FROM PropertyForRent p WHERE b.city = p.city);
```

4.3.10 Database Updates

SQL is a complete data manipulation language supporting both queries and database modifications. The three DML update statements are:

- **INSERT** — adds new rows of data to a table.
 - **UPDATE** — modifies existing data in a table.
 - **DELETE** — removes rows of data from a table.
-

Adding data to the database (INSERT)

The first form of INSERT inserts a single row:

```
INSERT INTO TableName [(columnList)]
```

```
VALUES (dataValueList)
```

TableName may be a base table or an updatable view. **columnList** is optional; if omitted, SQL assumes all columns in CREATE TABLE order. Omitted columns must have been declared nullable or with a DEFAULT value. The **dataValueList** must match **columnList** in number, position, and compatible data types.

EXAMPLE 4.35 — INSERT...VALUES

Insert a new row into the Staff table supplying data for all columns.

```
INSERT INTO Staff
```

```
VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', DATE '1957-05-25', 8300, 'B003');
```

All columns are supplied in CREATE TABLE order, so no column list is needed.

EXAMPLE 4.36 — INSERT using defaults

Insert a new row into the Staff table supplying data only for mandatory columns.

```
INSERT INTO Staff (staffNo, fName, lName, position, salary, branchNo)
```

```
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003');
```

Only the specified columns receive values; sex and DOB are implicitly set to NULL. An equivalent explicit formulation:

```
INSERT INTO Staff
```

```
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', NULL, NULL, 8100, 'B003');
```

The second form of INSERT copies multiple rows from other tables:

```
INSERT INTO TableName [(columnList)]
```

```
SELECT ...
```

The SELECT clause can be any valid SELECT statement. The same column compatibility rules apply.

EXAMPLE 4.37 — INSERT...SELECT

Populate a StaffPropCount table with staff names and the number of properties each manages.

```
INSERT INTO StaffPropCount
```

```
(SELECT s.staffNo, fName, lName, COUNT(*))
```

```
FROM Staff s, PropertyForRent p
```

```
WHERE s.staffNo = p.staffNo
```

```
GROUP BY s.staffNo, fName, lName)
```

```
UNION
```

```
(SELECT staffNo, fName, lName, 0
FROM Staff s
WHERE NOT EXISTS (SELECT *
FROM PropertyForRent p
WHERE p.staffNo = s.staffNo));
```

The first SELECT counts properties for staff who manage at least one. The UNION adds staff who manage none, with a count of 0. Without the UNION, staff with no properties would be excluded. The result is shown in Table 4.35.

staffNo	fName	lName	propCount
SG14	David	Ford	1
SL21	John	White	0
SG37	Ann	Beech	2
SA9	Mary	Howe	1
SG5	Susan	Brand	0
SL41	Julie	Lee	1

Modifying data in the database (UPDATE)

The UPDATE statement modifies existing rows:

```
UPDATE TableName
```

```
SET columnName1 = dataValue1 [, columnName2 = dataValue2 ...]
```

```
[WHERE searchCondition]
```

TableName can be a base table or an updatable view. The SET clause specifies which columns to change. If WHERE is omitted, all rows are updated. New values must be type-compatible with their columns.

EXAMPLE 4.38 — UPDATE all rows

Give all staff a 3% pay increase.

```
UPDATE Staff
```

SET salary = salary*1.03;

No WHERE clause — all rows are updated.

EXAMPLE 4.39 — UPDATE specific rows

Give all Managers a 5% pay increase.

UPDATE Staff

SET salary = salary*1.05

WHERE position = 'Manager';

Only rows satisfying the WHERE condition are updated.

EXAMPLE 4.40 — UPDATE multiple columns

Promote David Ford (staffNo = 'SG14') to Manager and change his salary to £18,000.

UPDATE Staff

SET position = 'Manager', salary = 18000

WHERE staffNo = 'SG14';

Deleting data from the database (DELETE)

The DELETE statement removes rows from a table:

DELETE FROM TableName

[WHERE searchCondition]

TableName can be a base table or an updatable view. If WHERE is omitted, all rows are deleted. The table definition remains — to delete both rows and the table structure, DROP TABLE must be used.

EXAMPLE 4.41 — DELETE specific rows

Delete all viewings that relate to property PG4.

DELETE FROM Viewing

WHERE propertyNo = 'PG4';

Only rows satisfying the condition are removed.

EXAMPLE 4.42 — DELETE all rows

Delete all rows from the Viewing table.

DELETE FROM Viewing;

No WHERE clause — all rows are removed. The table definition remains intact and the table can subsequently receive new data.

Chapter Summary

- SQL is a nonprocedural language composed of standard English words — SELECT, INSERT, DELETE, and others — that can be used by both technical and non-technical users. It is both the formal and de facto standard language for defining and manipulating relational databases.
- The SELECT statement is the most important command in the language. It combines the three fundamental relational algebra operations of Selection, Projection, and Join. Every SELECT statement produces a result table consisting of one or more columns and zero or more rows.
- The SELECT clause identifies the columns and calculated data to appear in the result. All column names in the SELECT clause must have their corresponding tables listed in the FROM clause.
- The WHERE clause restricts the rows included in the result by applying a search condition. The ORDER BY clause sorts the result on one or more columns, each in ascending or descending order. ORDER BY must always be the final clause of a SELECT statement.
- SQL provides five aggregate functions — COUNT, SUM, AVG, MIN, and MAX — that accept a column as input and return a single computed value. Mixing aggregate functions with unaggregated column references in a SELECT clause is illegal without a GROUP BY clause.
- The GROUP BY clause groups rows sharing the same values in specified columns, allowing aggregate functions to produce one summary value per group. The HAVING clause filters groups from the result in the same way WHERE filters individual rows; unlike WHERE, HAVING can include aggregate functions.
- A subselect is a complete SELECT statement embedded within another query. When it appears in a WHERE or HAVING clause, it is called a subquery or nested query.

Conceptually, a subquery produces a temporary table whose contents are used by the outer query. Three types exist: scalar (single value), row (multiple columns, single row), and table (multiple columns and rows).

- When the result table requires columns from more than one table, a join must be used by specifying multiple tables in the FROM clause. The ISO standard supports outer joins and the set operations UNION, INTERSECT, and EXCEPT.

- Beyond SELECT, the SQL DML provides INSERT — to add a single row or multiple rows from a subselect; UPDATE — to modify values in one or more columns of specified rows; and DELETE — to remove one or more rows from a table.